

Sudoku-Tool

Lena Marie Ludwig, Jari Hartmann, Lisa Rappold

Technische Hochschule Ulm

Juli 2026

Einleitung

Dieses Projekt hat das Ziel, ein Sudoku-Tool zu entwickeln, mit dem man interaktiv Sudoku spielen kann oder eine Funktion zur direkten Lösung des Rätsels bietet.

Das Programm kann ein Sudoku aus einer Textdatei einlesen und dieses auf Gültigkeit prüfen. Alternativ kann man ein Sudoku mit beliebiger Schwierigkeitsstufe generieren lassen. Anschließend kann der Benutzer entweder interaktiv das Sudoku lösen, oder das Sudoku mit einem von zwei Lösungsalgorithmen lösen lassen. Abschließend kann die Lösung des Sudokus gespeichert werden. In diesem Bericht stellen wir die Umsetzung und Funktionalität der einzelnen Funktionen vor.

1 main()

Die Funktion `main()` bildet den Einstiegspunkt des Programms. Sie bleibt bewusst kurz, ruft `callStartMenu()` auf und beendet das Programm nach dessen Rückkehr mit dem Rückgabewert 0. Die eigentliche Steuerung erfolgt über zwei Menüfunktionen. Im Startmenü wählt der User, ob ein Sudoku mit einer von drei Schwierigkeitsstufen erstellt oder aus `inputSudoku.txt` eingelesen werden soll. Ein eingelesenes Sudoku wird zunächst mit `validate()` geprüft. Ist es bereits vollständig, wird dies direkt ausgegeben. Andernfalls prüft `checkSolvability()` einmalig mithilfe des Donald-Knuth-Algorithmus, ob und wie oft das Sudoku lösbar ist. Danach führt `sudokuMenu()` zu den weiteren Funktionen. Der User kann das Sudoku selbst lösen, einen der beiden Solver starten oder sich einen Motivationsspruch anzeigen lassen. Nach einer automatischen Lösung kann das Ergebnis gespeichert werden. `getUserIntInput()` prüft sämtliche Menüeingaben und fängt Buchstaben sowie Werte außerhalb des jeweiligen Bereichs ab. Beim Verlassen gibt `returnToMenu()` den dynamisch reservierten Speicher der Matrix frei.

2 InteractiveSolver()

Das interaktive Lösen selbst wird in der Funktion von `playSudokuInteractive()` gesteuert. Vor jedem Spielzug wird das aktuelle Sudoku zusammen mit einem Aktionsmenü ausgegeben. Der User kann das Spiel abbrechen, eine Zahl eintragen oder einen Hinweis anfordern. `getUserInput()` erwartet Zeile, Spalte und Zahl als drei ganze Zahlen. Falsche Formate und Werte außerhalb von 1 bis 9 werden abgefangen. Für den Zugriff auf die Matrix werden die eingegebenen Koordinaten anschließend auf die Indizes 0 bis 8 umgerechnet. Ein Zug wird zuerst von `isLegalMove()` geprüft. Dabei muss das gewählte Feld leer sein und die Sudokueregeln dürfen nicht verletzt werden. Danach setzt `isStillSolvable()` den Wert nur testweise ein und prüft mit dem Donald-Knuth-Algorithmus, ob mindestens eine Lösung verbleibt. Erst wenn beide Prüfungen erfolgreich sind, wird die Zahl dauerhaft übernommen. Nach jedem Zug kontrolliert `isSudokuFinished()`, ob alle Felder belegt sind. In diesem Fall kann die Lösung gespeichert werden. Für einen Hinweis wird eine Kopie des aktuellen Sudokus mit dem Backtracking-Solver gelöst. Aus dieser Kopie kann eine Zeile, ein 3×3-Block oder ein einzelnes Feld angezeigt werden, ohne den Spielstand zu verändern. Diese Ausgaben werden bewusst nicht direkt ins Sudoku eingetragen, um den Rätselspaß nicht zu mindern.

3 read()

Eine `read()` Methode zum Einlesen von Sudoku-Rätseln ist essenziell für die Funktionalität des Tools. Hierfür gibt der User sein Sudoku in die Textdatei `inputSudoku.txt` ein. Anschließend wird versucht, die Textdatei im

Lesemodus zu öffnen. Ist das Öffnen erfolgreich, liest die Methode mithilfe einer while-Schleife das Sudoku ein. Hierbei werden kleine Fehler bei der Nutzereingabe ignoriert. Leerzeichen, senkrechte Striche, Bindestriche und leere Zeilen werden übersprungen. Bereits bei dem Einlesen wird eine grundlegende Validierung vorgenommen. Die Methode wirft eine Fehlermeldung, wenn in einer Zeile, die nicht leer ist, mehr oder weniger als 9 Zahlen vorkommen. Außerdem werden nur Eingaben von 0 bis 9 akzeptiert. Die Zahl 0 steht hierbei für ein leeres Feld. Auch bei zu vielen oder zu wenigen Zeilen wird eine Fehlermeldung ausgegeben. Handelt es sich bei dem eingegebenen Zeichen um eine Zahl zwischen 0 und 9, wird durch die Subtraktion des ASCII Wertes von 0 der korrekte Wert in der Matrix gespeichert. Zuletzt ruft die Funktion `printMatrix()` auf, um die eingelesene Matrix in der Konsole auszugeben.

4 create()

Alternativ kann man sich auch mithilfe von `create()` ein zufälliges, eindeutiges Sudoku generieren lassen. Hierfür werden zwei zufällige Zahlen in ein leeres Sudoku eingetragen. Diese stellen sicher, dass bei jedem Ausführen der Funktion unterschiedliche Sudokus generiert werden. Nachdem mithilfe der Funktion *DonaldKnuth* auf Gültigkeit geprüft wurde und eine Lösung für das Sudoku erstellt wurde, werden dann je nach Schwierigkeitsgrad mehr oder weniger Zahlen entfernt. Hierbei wird nach jeder entfernten Zahl geprüft, ob das Sudoku noch eindeutig lösbar ist, sonst wird eine andere Zahl entfernt.

5 validate()

Die Funktion `validate()` überprüft, ob das Sudoku gültig ist. Hierfür werden drei 9x9 Hilfs-Matrizen mit Nullen für Zeilen, Spalten und Blöcke erstellt.

Wird beispielsweise die erste Zeile des Sudokus überprüft, geht die Funktion jedes Element der ersten Zeile durch. Ist eine Zahl vorhanden, wird in der entsprechenden Zeile der Matrix *zeile* der Wert in Index *zahl-1* auf 1 gesetzt. Ist dieses Element in der Matrix schon eine 1, kommt diese Zahl doppelt vor und es wird eine Fehlermeldung ausgegeben.

Indem die Indizes der gleichen verschachtelten for-Schleifen getauscht werden, können die Spalten des Sudokus analog überprüft werden. Es wird mit der selben Logik vorgegangen. Hierbei repräsentiert jede Spalte in der Matrix *spalte* eine Spalte in dem Sudoku.

Das Validieren der Blöcke ist etwas komplexer. Hierfür werden zwei Hilfsarrays `BLOCKZEILE` und `BLOCKSPALTE` erstellt, die für jeden der neun Blöcke die Koordinaten der linken oberen Ecke speichern. Die Zeile wird ermittelt, indem der entsprechende Wert aus `BLOCKZEILE` mit $j/3$ addiert wird. Da Nachkommastellen von Integern abgeschnitten werden, werden die entsprechenden Werte in `BLOCKZEILE` mit 0, 1 und 2 addiert, was die korrekte Zeile ergibt.

Die Spalte wird ermittelt, indem der entsprechende Wert aus `BLOCKSPALTE` mit $j\%3$ addiert wird. Hierdurch wird `BLOCKZEILE` mit der Sequenz 0, 1, 2 addiert, was die korrekte Spalte ergibt.

Hierbei wird jeder Block in der Matrix *block* in einer Zeile gespeichert. Auch hier repräsentiert jede Spalte eine Zahl.

6 Solver mittels Backtracking

Ein wesentliches Ziel dieser Projektarbeit bestand darin, einen Solver zu entwickeln, der für ein eingegebenes oder selbst generiertes Sudoku eine valide Lösung berechnet. Ein naheliegender und simpler Ansatz hierfür ist das *Backtracking* Prinzip [3]. Das Sudoku Spielfeld lässt sich als ein graphischer Baum interpretieren, in dem jedes der 81 einzelnen Felder eine Ebene bzw. Generation des Baums repräsentiert. Für jede mögliche Ziffer 1 bis 9 existiert ein Knoten im Baum, der wiederum neun Kinderknoten besitzt, welche die möglichen Ziffern des jeweils nachfolgenden Felds im Sudoku darstellen. Es entsteht also ein Graph aus insgesamt 9^{81} Knoten, wobei anzumerken ist, dass es sich dabei lediglich um ein theoretisches Konstrukt zur Veranschaulichung des Algorithmus handelt. Es wurde also kein Baum programmiertechnisch umgesetzt, was allerdings zur Optimierung des Codes noch einen möglichen Ansatzpunkt hätte darstellen können.

Im Backtracking Algorithmus jedenfalls wird der beschriebene Baum *rekursiv* nach einer Lösung des Sudokus durchsucht. Dies geschieht nach dem Prinzip der *Tiefensuche*: Für eine zu testende Ziffer auf Ebene *i* des Baums - entsprechend dem *i*-ten Feld des Sudoku Rätsels - werden zunächst alle Möglichkeiten auf Ebene *i + 1* - entsprechend dem nachfolgenden Kästchen im Sudoku-Gitter - überprüft. Erst wenn auf Ebene *j* für

jede der Ziffern 1 bis 9 ein Fehler auftritt - wenn also für das Feld j keine gültige Besetzung existiert auf Basis der vorangegangenen Felder - dann erst wird das Feld j auf den Eintrag 0 zurückgesetzt und im Solver auf Ebene $j - 1$ zurück gesprungen, um die nächste Ziffer auszuprobieren. Ein Fehler besteht dabei in einer *Kollision*, also dem mehrfachen Auftreten derselben Ziffer in der gleichen Zeile oder Spalte bzw. im gleichen 3×3 Block. Solch ein Fehler wird durch die Funktion `validate_cell()` entdeckt, die in jedem Schritt des Solvers aufgerufen wird.

Weiterhin ruft die `solve()` Funktion sich immer wieder selbst auf: Sie besitzt nämlich neben dem Rätselgitter `gitter` auch die Eingabe Parameter `int Zeile` und `int Spalte`, welche die Position des zu lösenden Kästchens im Sudokugitter spezifizieren. Im Rahmen der Tiefensuche wird also `solve(Zeile, Spalte+2, gitter)` bzw. am Ende einer Zeile `solve(Zeile+1, 0, gitter)` aufgerufen, sodass die Kästchen von links oben nach rechts unten zeilenweise durchlaufen werden. Bereits vorgegebene Felder, in denen also beim Einlesen keine 0 steht, werden natürlich übersprungen. Wie in jeder Rekursion muss auch bei der Tiefensuche im Sudokugitter ein *Basisfall* definiert werden: Ist bis zum Aufruf `solve(8,8, gitter)` keine Kollision aufgetreten, so wurde eine vollständige Lösung gefunden und es erfolgt nur noch der *rekursive Aufstieg*, bei dem der nach oben weitergereichte Rückgabe `return 1`; den Sucherfolg anzeigt.

7 Erweiterung: 4x4 Sudokus

Um das Tool noch etwas vielseitiger zu gestalten, wurde es um die Möglichkeit zum Lösen von *Hexadezimal Sudokus* ergänzt. Bei dieser Sonderform des Rätselspiels wird das 9×9 Spielfeld zu einem 16×16 Feld erweitert und es wird mit den Hexadezimal Ziffern 0–9 & A-F gearbeitet. Hier kann also die Ziffer 0 nicht mehr wie in Abschnitt 3 beschrieben für ein unbesetztes Feld stehen, sondern dafür wird nun das Makro `EMPTY` mit dem Wert `-1` definiert und die Funktionen zum Einlesen und Ausgeben des Rätsels entsprechend angepasst. Der Solver arbeitet intern weiterhin mit dem Datentyp `Integer` und die Ziffern werden lediglich zur Ausgabe in die Hexadezimal Ziffern umgewandelt. Darauf könnte man im Weiteren verzichten, wenn die Erweiterung für beliebig große Sudokus genutzt werden können soll. Um diese potentielle Folgeversion zu erleichtern, wurde bereits im 4×4 Sudoku mit dem Makro `SIZE` anstelle einer statischen Größe.

Schon für `SIZE=4` ergibt sich allerdings im Backtracking eine Größe des Suchbaums von $16^{16 \cdot 16}$ Knoten, was bereits bei mittelschweren Sudokus zu unzumutbaren Laufzeiten führt. Damit verdeutlicht diese Erweiterung auch recht eindrücklich die wahrscheinlich wichtigste Limitation am Backtracking-Algorithmus: Durch das *Brute Force* Prinzip, also das systematische Ausprobieren aller potentiellen Lösungen, wächst die Laufzeit rapide an, im Falle des Sudokus nämlich exponentiell. Um auch größere Sudokus bzw. Sudokus mit einem geringen Anteil an vorgegebenen Zahlen in einer zufriedenstellenden Laufzeit lösen zu können, ist deshalb eine andere Strategie erforderlich.

8 solve() mithilfe des Algorithm X

Alternativ kann ein Sudoku auch als Exact-Cover-Problem betrachtet und mit einer vereinfachten Version des Algorithm X von Donald Knuth gelöst werden. Hierfür wird mithilfe von `calloc()` und `malloc()` eine Matrix der Größe 729×324 erstellt. In den Zeilen dieser Matrix werden alle möglichen Züge repräsentiert. In den Spalten werden für jeden Zug die vier grundlegenden Sudoku Regeln gespeichert: Jedes Feld muss belegt sein, in jede Zeile und Spalte kommt jede Zahl einmal vor, und pro 3×3 Block kommt jede Zahl genau einmal vor. Am Ende sind also in jeder Zeile vier Einsen (abgesehen von vorgegeben Feldern).

Außerdem wird ein Array `coveredColumns` eingeführt, welches die bereits erfüllten Regeln speichert. Ist in jedem Index dieses Arrays genau eine Eins, ist das Sudoku valide gelöst. Es gibt auch ein Array `solution`, welches die verwendeten Zeilen von der großen Matrix speichert. Diese werden am Ende für das Zurückübersetzen der Binärarrays in die Lösung des Sudokus benötigt.

Eine Funktion `solveMatrix()` nutzt dann Rekursion und Backtracking, um eine Lösung zu finden. Hierbei werden bei jeder Null in `coveredColumns` alle gültigen Spielzüge gezählt, wobei gezielt nach der Spalte mit den wenigsten möglichen Spielzügen gesucht wird, um ein frühzeitiges Erkennen von Sackgassen zu ermöglichen. Wird eine Zeile gefunden, wird diese in `solution` und `coveredColumns` gespeichert. Falls `coveredColumns` noch nicht ausschließlich mit Einsen gefüllt ist, aber trotzdem keine passende Regel vorhanden ist, wird der letzte Spielzug rückgängig gemacht und die Rekursion läuft weiter. Je nach Modus kann das Sudoku entweder normal gelöst werden oder über eine Funktion `countSolutionsMatrix()` die Anzahl der Lösungen ermitteln.

9 output()

Diese Funktion hat das Ziel, das Sudoku formatiert in einer Textdatei zu speichern.

Hierfür öffnet sie zunächst die Zielfeile *outputSudoku.txt* im Schreibmodus. Schlägt dies fehl, wird eine Fehlermeldung ausgegeben. Für die Ausgabe der Matrix wird eine for-Schleife über insgesamt elf Durchläufe genutzt, wobei neun für die Zeilen des Sudokus stehen und zwei für die Trennstriche zwischen den 3x3 Blöcken. Hierbei gibt es die Variable *verschiebung*, die dafür sorgt, dass trotz der Trennstriche die korrekten Zeilen der Matrix ausgegeben werden. Das erfolgt, indem die Variable jeweils bei dem Einfügen der horizontalen Trennstrichen hochgezählt wird. Dann wird jeweils die Matrix in der Zeile *i-verschiebung* ausgegeben. Die Trennstriche werden bei den Durchläufen 3 und 7 ausgegeben. Die vertikalen Abtrennungen der 3x3 Blöcke werden manuell durch senkrechte Striche in der Ausgabe eingefügt. Nach der Ausgabe wird die Datei wieder geschlossen.

10 Tests

Die automatisierten Tests befinden sich in *testSudoku.c* und werden über eine eigene *main()*-Funktion gestartet. Die Tests zielen besonders auf randfälle und falsche eingaben des Users ab. Die Hilfsfunktion *pruefeTest()* vergleicht für jeden Test das erhaltene mit dem erwarteten Ergebnis. Weitere Hilfsfunktionen erstellen und löschen die dynamischen Testmatrizen oder prüfen, ob ein gelöstes Sudoku vollständig ausgefüllt ist. Insgesamt werden 17 Einzeltests ausgeführt. *validate()* wird mit einem gültigen Sudoku sowie mit doppelten Zahlen in Zeile, Spalte und 3x3-Block getestet. Für *validate_cell()* werden ein erlaubter Wert und Konflikte in allen drei Bereichen geprüft. Beide Solver müssen eine Lösung finden, diese vollständig ausfüllen und ein gültiges Sudoku erzeugen. Beim Donald-Knuth-Algorithmus wird zusätzlich die eindeutige Lösbarkeit kontrolliert. *createSudoku()* muss ein gültiges und eindeutig lösbares Sudoku erstellen. Als Randfall wird außerdem geprüft, ob *read()* eine leere Eingabedatei korrekt abfängt. Eine vorhandene Datei wird dafür vorübergehend gesichert und anschließend wiederhergestellt. Erwartete Fehlerausgaben der Funktionen selbst werden abhängig vom Betriebssystem nach NUL beziehungsweise */dev/null* umgeleitet, dies dient der besseren Übersichtlichkeit. Sind alle Tests bestanden, beendet sich das Testprogramm mit 0, andernfalls mit 1. Da es eine eigene *main()*-Funktion besitzt, darf *main.c* beim Kompilieren der Tests nicht eingebunden werden. In der finalen Version wurden alle 17 Tests erfolgreich bestanden.

Zusammenarbeit mit git und github

Neben der Implementierung des Sudoku Programms lag ein wesentliches Ziel der Projektarbeit darin, das Versions-Kontrollsystem GIT kennenzulernen und den GITHUB workflow zu verinnerlichen. Genau darin bestand auch eine große Herausforderung des Projekts, denn für Beginner erfordert der Einstieg in GIT einige Umstellungen im Arbeitsprozess des Programmierens und es müssen sich zusätzliche Routinen angeeignet werden. Durch das 4-Augen-Prinzip bei der Annahme von *pull-requests* kann jedoch ein erheblicher Mehrwert für die Qualitätssicherung des Codes erreicht werden. Gegenseitige Kommentare und Verbesserungsvorschläge am Code der anderen Teammitglieder sowie Diskussionen über allfällige *merge-Konflikte* intensivieren zudem den stetigen Austausch im Team. Die Nutzung verschiedener *Branches* erlaubt das gleichzeitige Bearbeiten eines Projektes an verschiedenen Angriffspunkten und durch mehrere Teammitglieder.

Im Rahmen der Projektarbeit konnten hiermit erste Erfahrungen gesammelt werden und einige Vorzüge der Nutzung eines Systems wie GIT konnten trotz aller Schwierigkeiten beim Erlernen des workflows bereits erlebt werden. Weiterhin hat uns das Projekt gezeigt, wie sich der modulare Aufbau eines Programms bewähren kann und wie wichtig es in der Softwareentwicklung ist, neben dem eigenen Programmieren auch den Code anderer Teammitglieder detailliert nachzuvollziehen.

Fazit

Die Implementierung des Sudoku-Tools konnte erfolgreich abgeschlossen und durch das Bestehen aller 17 finalen Tests vollständig verifiziert werden. Mit der fertigen Anwendung steht nun ein vielseitiges Werkzeug zur Verfügung, das es dem Nutzer ermöglicht, Sudokus sowohl interaktiv über die Benutzeroberfläche zu lösen als auch programmgestützt valide Rätsel generieren zu lassen.

Aufteilung

- **Lisa Rappold:** input(), output(), validate(), create(), Solver nach Donald Knuth Algorithmus
- **Lena Ludwig:** Backtracking Solver, 4x4 Sudokus, Teile des Sudokus anzeigen, zufällige Motivationsprüche
- **Jari Hartmann:** Mainmethode, interactiveSolver(), Tests

Literatur

- [1] Sudoku Garden. Sudoku mit dancing links (dlx) lösen. <https://sudokugarden.de/de/loesen/dancing-links>. Abgerufen am 10.07.2026.
- [2] Shivan Kaul. Solving sudoku using knuth's algorithm x, 2021. Abgerufen am 10.07.2026.
- [3] mamins1376. Github repository. <https://gist.github.com/2066828739627d15c112dd4ec582db46>.git, 2016.
- [4] stackoverflow. The dancing links algorithm - an explanation that is less explanatory but more on implementation? <https://stackoverflow.com/questions/1518335/the-dancing-links-algorithm-an-explanation-that-is-less-explanatory-but-more-o>. Abgerufen am 10.07.2026.